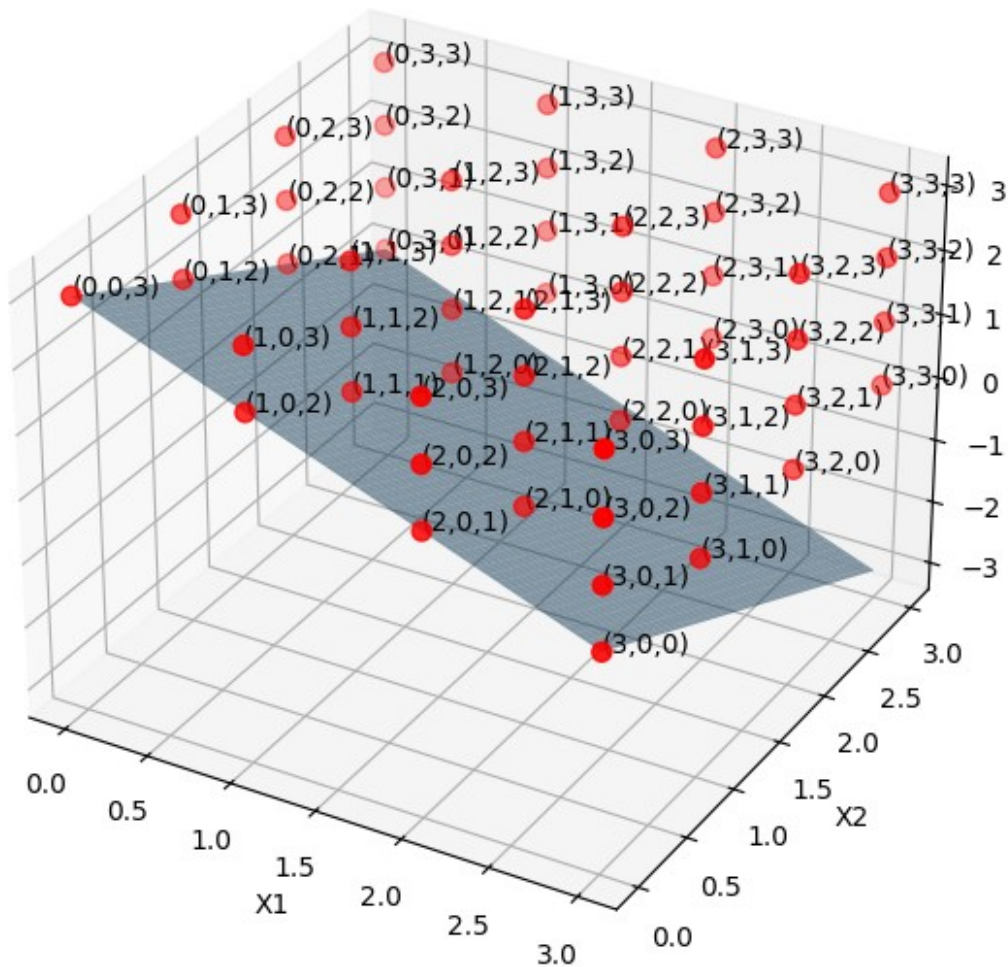


Q1. Apply Mc-Culloch Pits model for the following Boolean operations: Plot the decision boundaries for 2 - dimensional and 3-dimensional. Implement AND with 3-variables and threshold 3.

```
import numpy as np
import matplotlib.pyplot as plt
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')
x1 = np.linspace(0, 3, 100)
x2 = np.linspace(0, 3, 100)
X1, X2 = np.meshgrid(x1, x2)
X3 = 3 - X1 - X2
ax.plot_surface(X1, X2, X3, alpha=0.5)
points = np.array([(i, j, k) for i in range(4) for j in range(4) for k
in range(4) if i + j + k >= 3])
ax.scatter(points[:, 0], points[:, 1], points[:, 2], color='r', s=50)
for point in points:
    ax.text(*point, f'({point[0]},{point[1]},{point[2]})',
color='black')
ax.set_xlabel('X1')
ax.set_ylabel('X2')
ax.set_zlabel('X3')
ax.set_title('3D AND Operation with Threshold 3')
plt.show()
```

3D AND Operation with Threshold 3

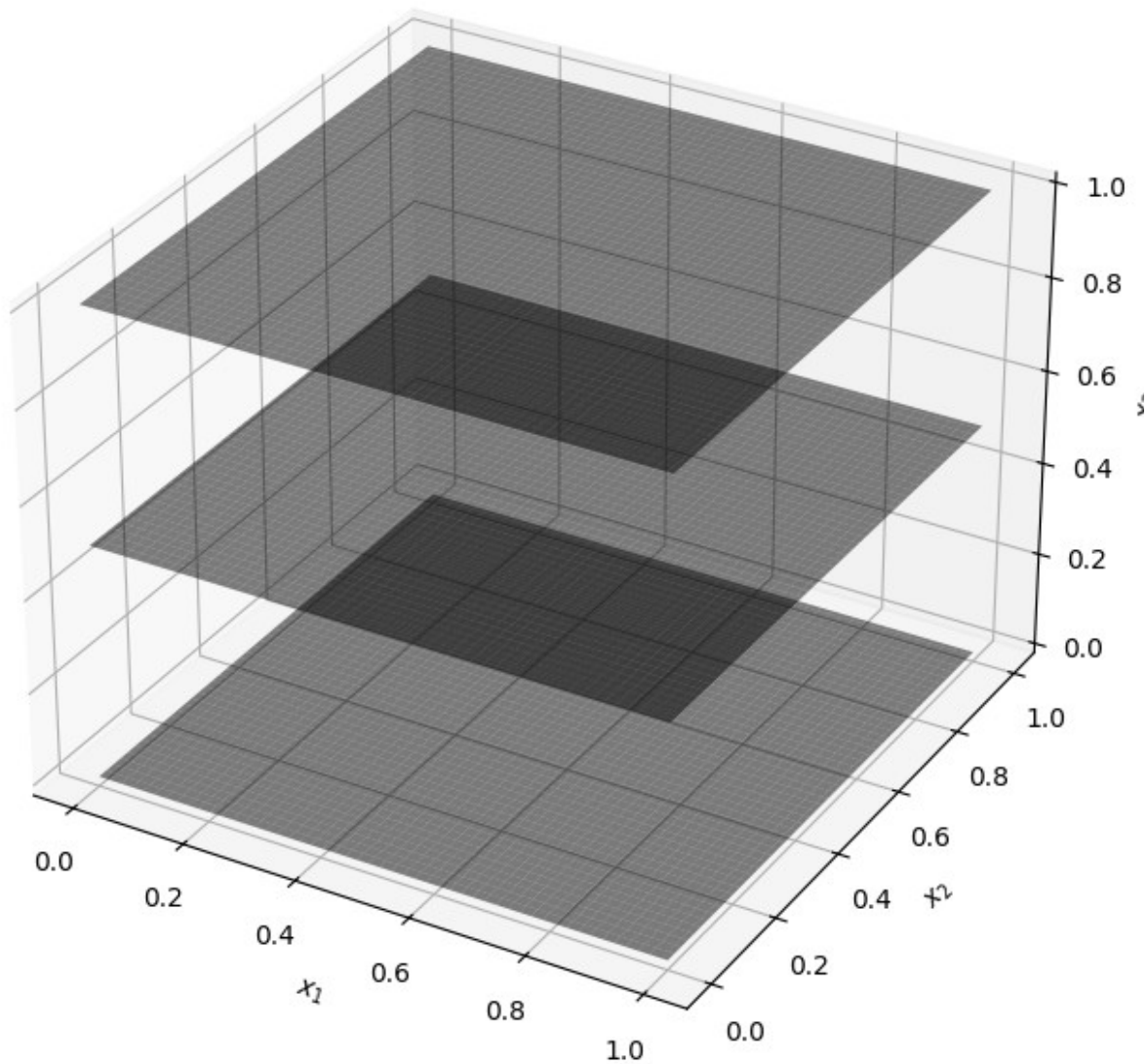


Implement OR with 3-variables and threshold 1.

```
x = np.linspace(0, 1, 100)
X1, X2 = np.meshgrid(x, x)
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
x3_values = [0, 0.5, 1]
for x3 in x3_values:
    Z = (X1 + X2) >= (1 - x3)
    ax.plot_surface(X1, X2, x3 * np.ones_like(X1),
                    facecolors=plt.cm.gray(Z), alpha=0.5,
                    edgecolor='none')
ax.set_xlabel('$x_1$')
ax.set_ylabel('$x_2$')
ax.set_zlabel('$x_3$')
ax.set_title('Decision Boundary for 3-Variable OR Operation with
```

```
Threshold 1')  
plt.show()
```

Decision Boundary for 3-Variable OR Operation with Threshold 1



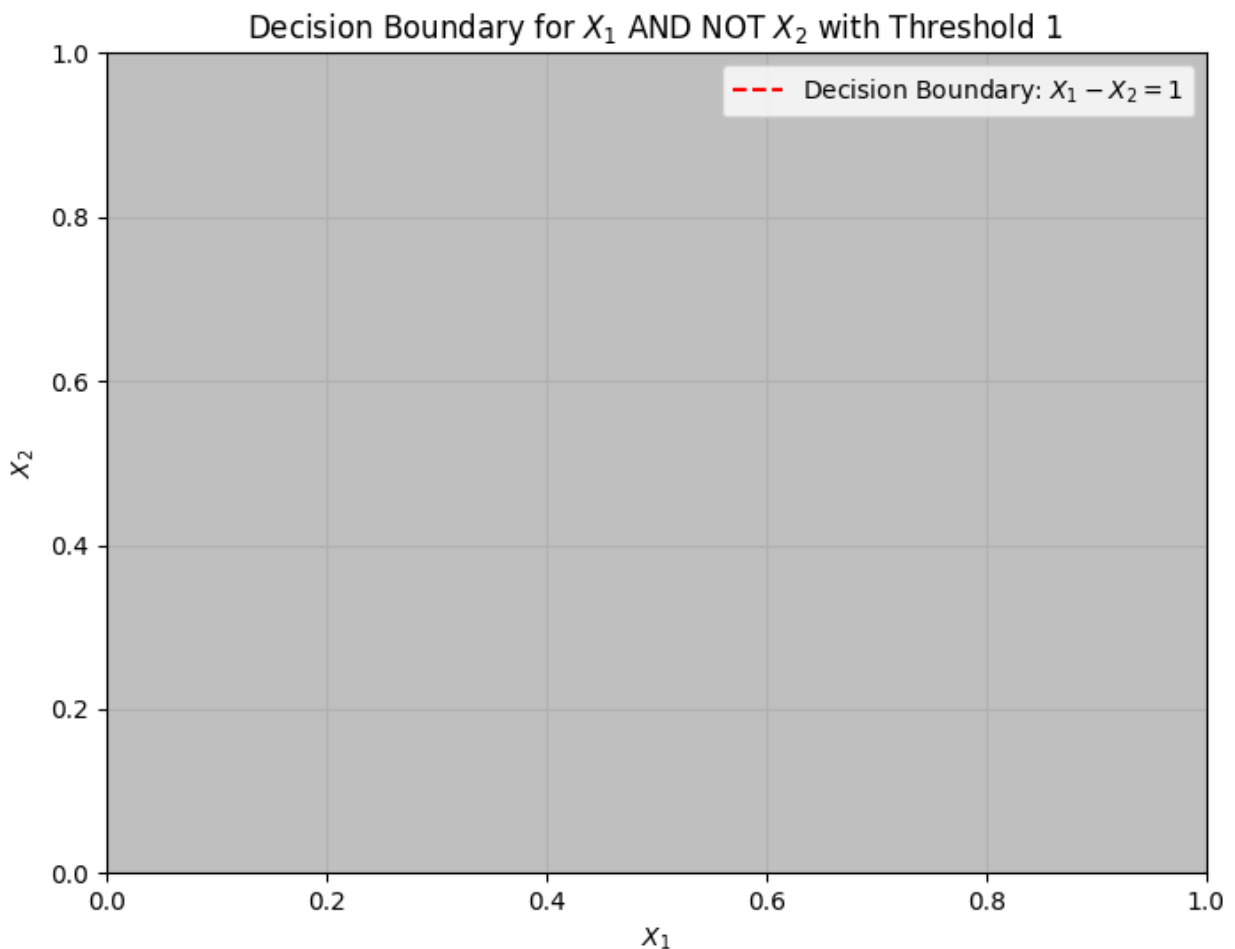
Implement X_1 and X_2 with 2-variables and threshold 1.

```
x1 = np.linspace(0, 1, 100)  
x2 = np.linspace(0, 1, 100)  
X1, X2 = np.meshgrid(x1, x2)  
weighted_sum = X1 - X2  
decision_boundary = weighted_sum >= 1  
plt.figure(figsize=(8, 6))
```

```

plt.contourf(X1, X2, decision_boundary, alpha=0.5, cmap='gray',
levels=[0, 1])
plt.plot(X1[0, :], X1[0, :] - 1, 'r--', label='Decision Boundary:  $X_1 - X_2 = 1$ ')
plt.xlabel('$X_1$')
plt.ylabel('$X_2$')
plt.title('Decision Boundary for  $X_1$  AND NOT  $X_2$  with Threshold 1')
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(True)
plt.legend()
plt.show()

```



Implement NOR with 2-variables and threshold 0.

```

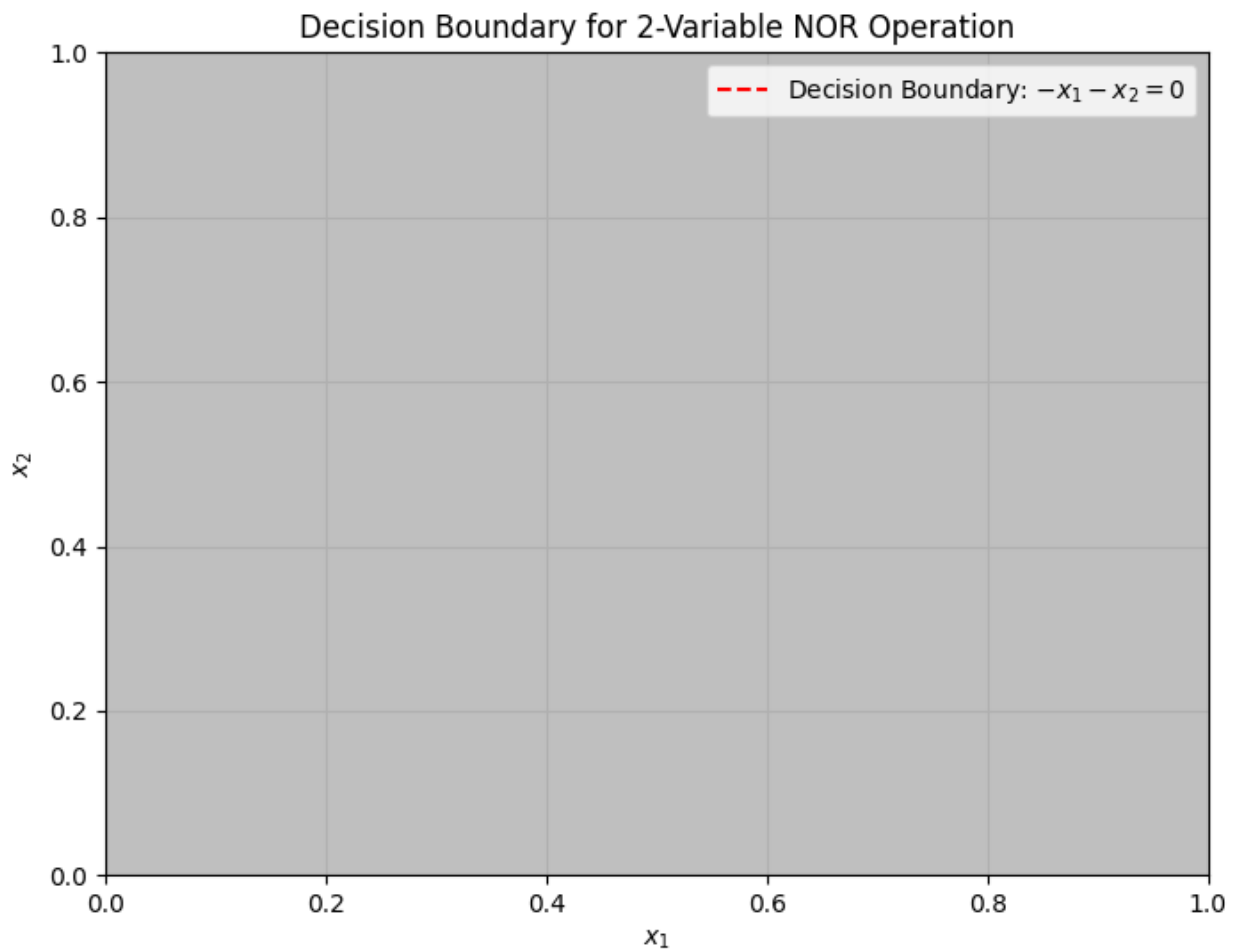
x1 = np.linspace(0, 1, 100)
x2 = np.linspace(0, 1, 100)

```

```

X1, X2 = np.meshgrid(x1, x2)
weighted_sum = -X1 - X2
decision_boundary = weighted_sum >= 0
plt.figure(figsize=(8, 6))
plt.contourf(X1, X2, decision_boundary, alpha=0.5, cmap='gray',
levels=[0, 1])
plt.plot(X1[0, :], -X1[0, :] + 0, 'r--', label='Decision Boundary: $-x_1 - x_2 = 0$')
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.title('Decision Boundary for 2-Variable NOR Operation')
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(True)
plt.legend()
plt.show()

```

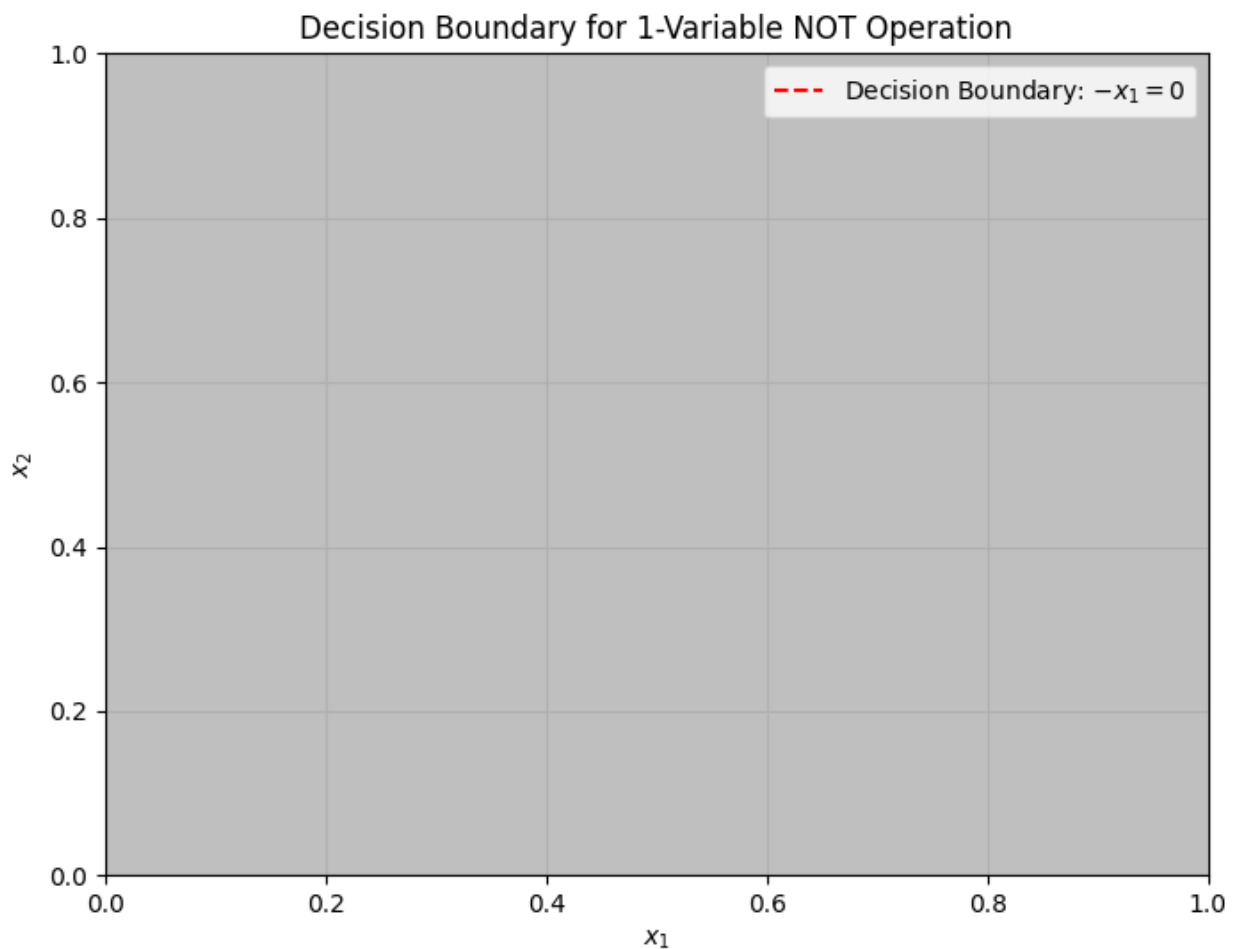


Implement NOT with 2-variables and threshold 0.

```

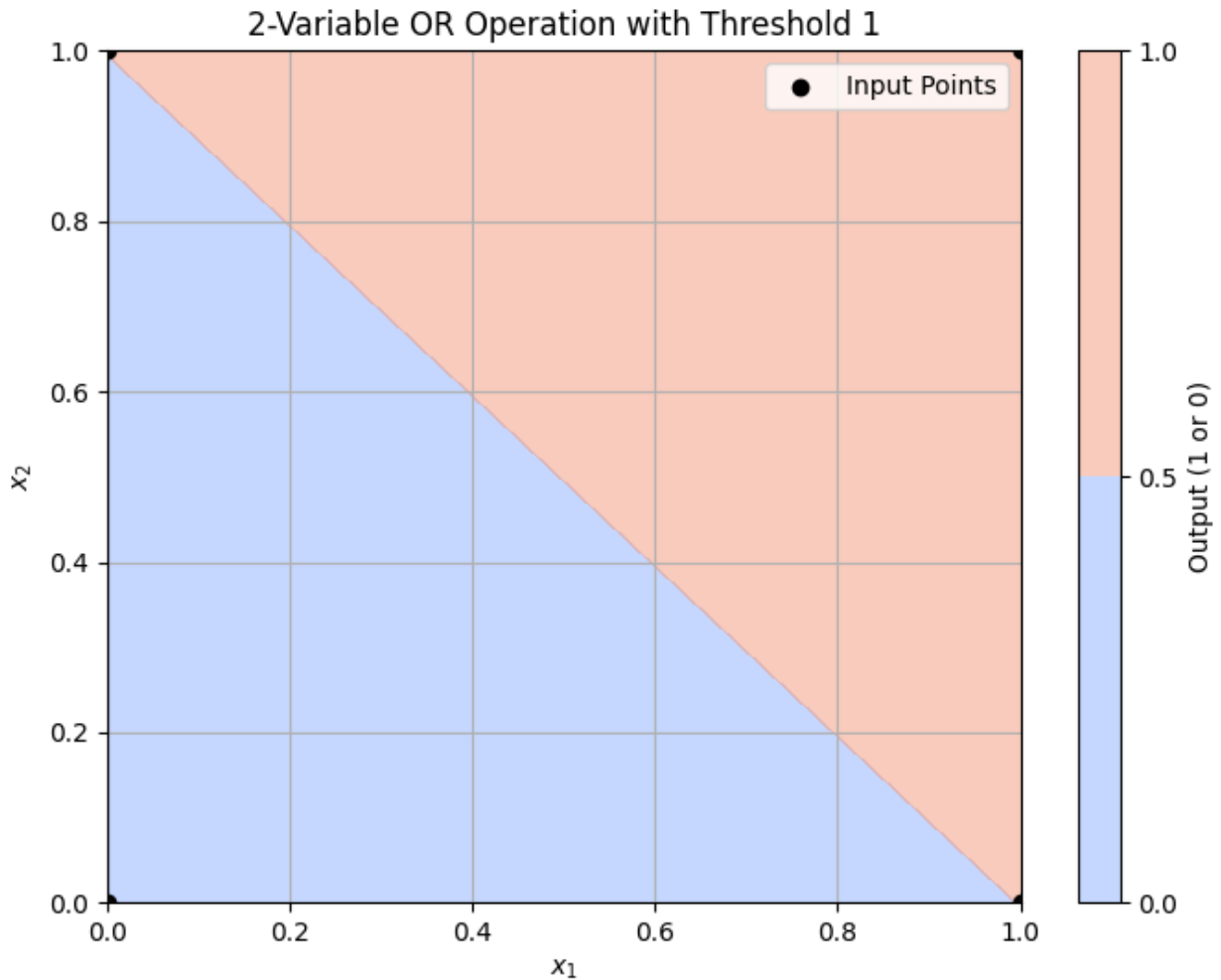
x1 = np.linspace(0, 1, 100)
x2 = np.linspace(0, 1, 100)
X1, X2 = np.meshgrid(x1, x2)
weighted_sum = -X1
decision_boundary = weighted_sum >= 0
plt.figure(figsize=(8, 6))
plt.contourf(X1, X2, decision_boundary, alpha=0.5, cmap='gray',
levels=[0, 1])
plt.plot(X1[0, :], -X1[0, :] + 0, 'r--', label='Decision Boundary: $-x_1 = 0$')
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.title('Decision Boundary for 1-Variable NOT Operation')
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(True)
plt.legend()
plt.show()

```



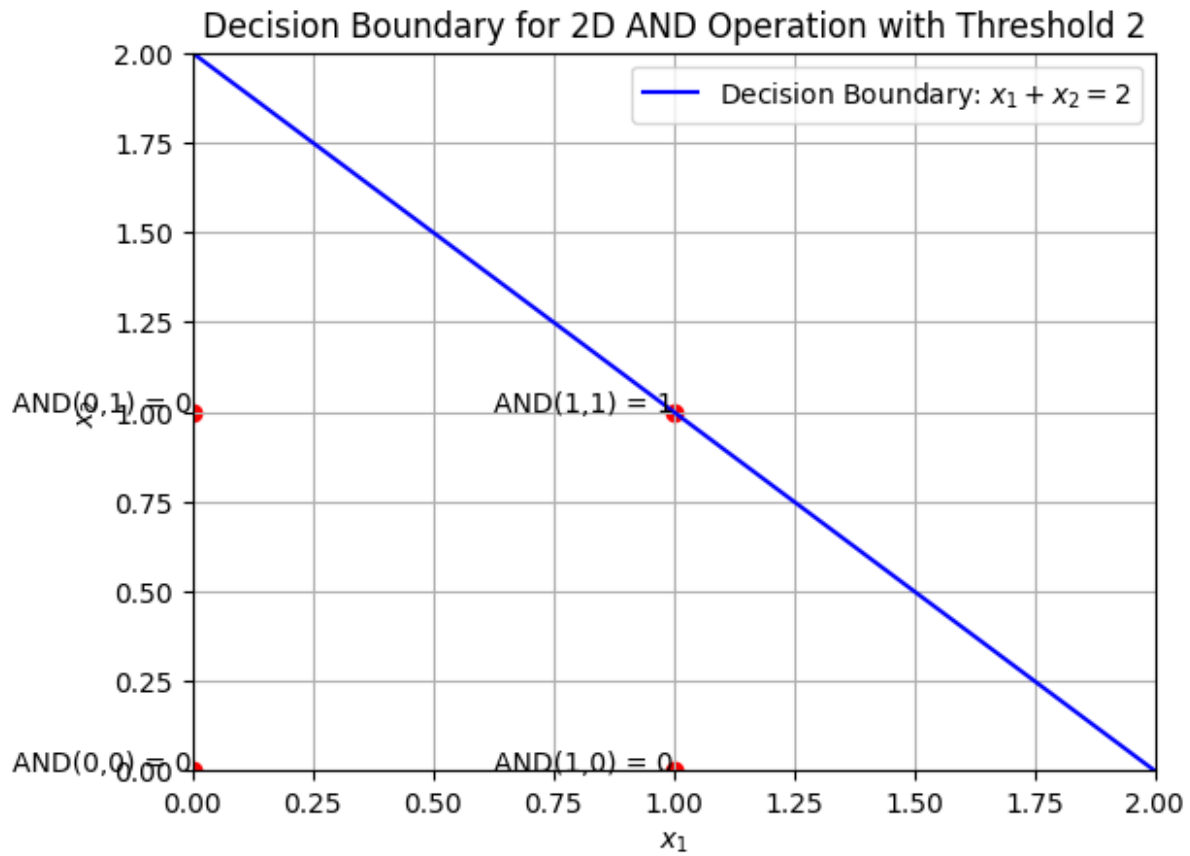
Implement OR with 2-variables and threshold 1.

```
def or_gate(x1, x2):  
    return 1 if (x1 + x2) >= 1 else 0  
  
# Generate input values for x1 and x2  
input_combinations = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])  
  
# Calculate the output for each combination of x1 and x2  
outputs = np.array([or_gate(x1, x2) for x1, x2 in input_combinations])  
  
# Display the results  
for (x1, x2), output in zip(input_combinations, outputs):  
    print(f"OR({x1}, {x2}) = {output}")  
  
# Plotting the decision boundary  
# Create a grid for x1 and x2  
x1 = np.linspace(0, 1, 100)  
x2 = np.linspace(0, 1, 100)  
X1, X2 = np.meshgrid(x1, x2)  
  
# Calculate the decision boundary  
Z = (X1 + X2) >= 1  
  
# Plotting  
plt.figure(figsize=(8, 6))  
plt.contourf(X1, X2, Z, alpha=0.5, cmap='coolwarm')  
plt.colorbar(label='Output (1 or 0)')  
plt.scatter(input_combinations[:, 0], input_combinations[:, 1],  
            color='black', label='Input Points')  
  
# Set labels and title  
plt.xlabel('$x_1$')  
plt.ylabel('$x_2$')  
plt.title('2-Variable OR Operation with Threshold 1')  
plt.xlim(0, 1)  
plt.ylim(0, 1)  
plt.axhline(0, color='black', lw=0.5, ls='--')  
plt.axvline(0, color='black', lw=0.5, ls='--')  
plt.legend()  
plt.grid()  
  
# Show the plot  
plt.show()  
  
OR(0, 0) = 0  
OR(0, 1) = 1  
OR(1, 0) = 1  
OR(1, 1) = 1
```



Implement AND with 2-variables and threshold 2.

```
x1 = np.linspace(0, 2, 100)
x2 = 2 - x1
plt.plot(x1, x2, label='Decision Boundary:  $x_1 + x_2 = 2$ ',
color='blue')
plt.scatter([0, 0, 1, 1], [0, 1, 0, 1], color='red')
plt.text(0, 0, 'AND(0,0) = 0', fontsize=10, ha='right')
plt.text(1, 0, 'AND(1,0) = 0', fontsize=10, ha='right')
plt.text(0, 1, 'AND(0,1) = 0', fontsize=10, ha='right')
plt.text(1, 1, 'AND(1,1) = 1', fontsize=10, ha='right')
plt.xlim(0, 2)
plt.ylim(0, 2)
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.title('Decision Boundary for 2D AND Operation with Threshold 2')
plt.legend()
plt.grid(True)
plt.show()
```

Implement OR with 3-variables and threshold 1.

```
x = np.linspace(0, 1, 100)
X1, X2 = np.meshgrid(x, x)
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
x3_values = [0, 0.5, 1]
for x3 in x3_values:
    Z = (X1 + X2) >= (1 - x3)
    ax.plot_surface(X1, X2, x3 * np.ones_like(X1),
                    facecolors=plt.cm.gray(Z), alpha=0.5,
                    edgecolor='none')
ax.set_xlabel('$x_1$')
ax.set_ylabel('$x_2$')
ax.set_zlabel('$x_3$')
ax.set_title('Decision Boundary for 3-Variable OR Operation with
Threshold 1')
plt.show()
```

Decision Boundary for 3-Variable OR Operation with Threshold 1

